



National University of Computer & Emerging Sciences -
NUCES - Karachi
Department of Artificial Intelligence

Computer Vision

AI-4002

Lab 05

Wavelets, Boundary
Detection, Hough Transform
& SIFT

Talha Shahid

Course Code: AI-4002	Computer Vision Lab
-----------------------------	----------------------------



Contents

1. Objectives	3
2. Introduction to Wavelet Transformation	3
2.1 Continuous Wavelet Transform (CWT).....	3
2.2 Discrete Wavelet Transform (DWT).....	3
2.3 Wavelet Packet Transform (WPT).....	4
2.4 Inverse Wavelet Transform	4
3. Boundary Detection.....	5
3.1 Sobel Edge Detection.....	6
3.2 Canny Edge Detection	11
3.3 Laplacian of Gaussian (LoG)	14
4. Introduction to Hough Transformation	17
4.1 Hough Transform for Line Detection.....	17
4.2 Hough Transform for Circle Detection	18
5. SIFT Feature Extraction	18
5.1 Scale-Space Extrema Detection	18
5.2 Keypoint Localization	19
5.3 Orientation Assignment	20
5.4 Keypoint Descriptor Generation.....	20
5.5 Keypoint Matching	21
5.6 Outlier Rejection	21
5.7 Final Matching and Homography Estimation.....	22



1. Objectives

1. Wavelet Transformation
2. Boundary Detection
3. Hough Transformation
4. Line Detection by Hough Transformation
5. SIFT Feature Extraction

2. Introduction to Wavelet Transformation

Wavelet transformation is a mathematical technique used in signal processing, data analysis, and image processing. It allows you to analyze a signal or image in terms of its frequency components at different scales. Wavelet transforms provide a time-frequency analysis that is particularly useful for non-stationary signals. There are several types of wavelet transforms, with the most common being the Continuous Wavelet Transform (CWT) and the Discrete Wavelet Transform (DWT).

2.1 Continuous Wavelet Transform (CWT)

Mathematical Function: The CWT involves continuous wavelet functions that are dilated and translated across the signal. The wavelet function, often denoted as $\psi(t)$, is scaled by different frequencies and translated across time.

Wavelet Families: Different wavelet families can be used, such as the Morlet wavelet for analyzing oscillatory components in signals.

Applications: CWT is useful for analyzing time-frequency properties of continuous signals and is commonly used in applications like seismic signal analysis and image analysis.

2.2 Discrete Wavelet Transform (DWT)

Mathematical Function: The DWT involves discrete wavelet functions that are used to transform discrete signal data. The wavelet function is scaled and shifted across the signal in a discrete manner.

Wavelet Families: DWT uses various wavelet families, including Daubechies, Haar, and Symlet wavelets.

Applications: DWT is widely used in image compression, image denoising, and feature extraction. It's also used in signal compression and analysis.



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

2.3 Wavelet Packet Transform (WPT)

Mathematical Function: Wavelet packet transform is an extension of the DWT that allows for more flexible decomposition of signals. It decomposes signals into multiple subbands.

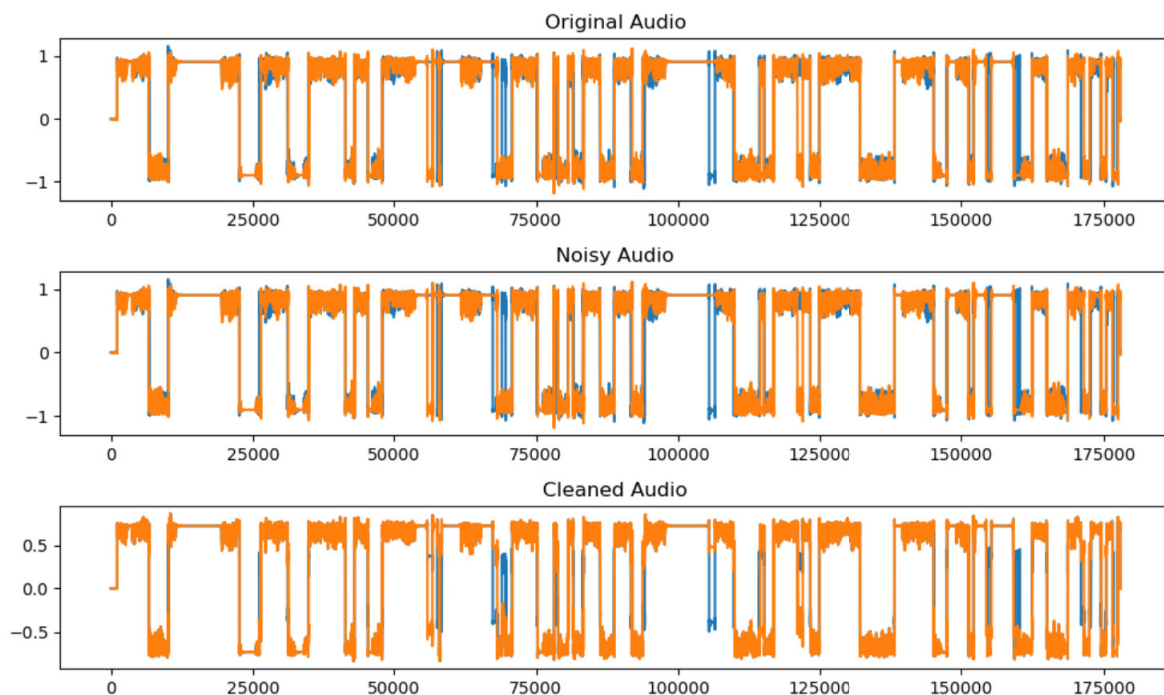
Applications: WPT can be used for analyzing signals with complex frequency content and for feature extraction in machine learning.

2.4 Inverse Wavelet Transform

Mathematical Function: To reconstruct a signal or image from its wavelet coefficients, an inverse wavelet transform is applied. It uses the original data along with the transformed coefficients to reconstruct the original signal.

Applications: Inverse wavelet transforms are crucial for signal and image reconstruction in applications like image compression and de-noising.

Talha Shahid



3. Boundary Detection

Boundary detection, often referred to as edge detection, is a fundamental concept in computer vision. It plays a crucial role in identifying the boundaries of objects within an image. These boundaries represent significant changes in intensity or color and are essential for various computer vision tasks, such as object recognition, image segmentation, and feature extraction.

Edges in an image are critical because they contain important information about the shapes and structures present. By detecting edges, computer vision algorithms can focus on these key regions, making it easier to analyze and interpret the content of an image.

BOUNDARY DETECTION (EDGE DETECTION)



Input: Original Scene (Intensity & Color)



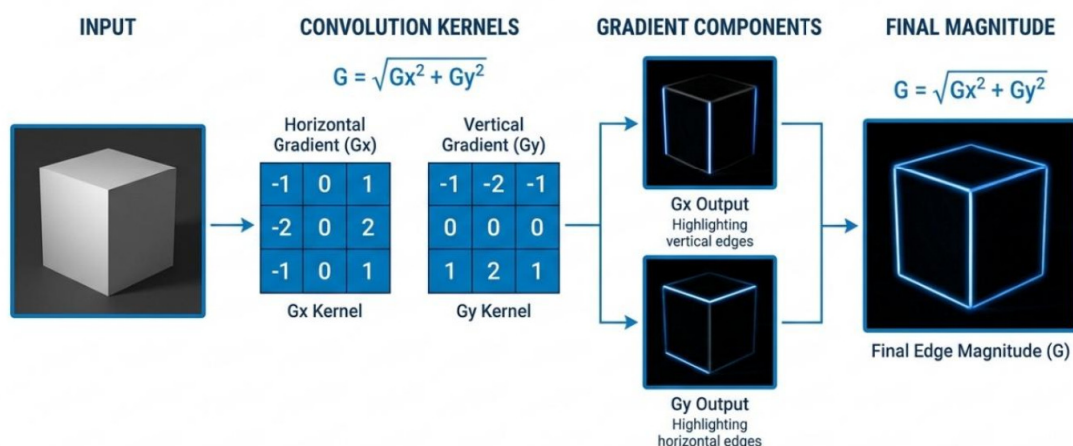
Output: Edge Map (Significant Changes in Intensity)

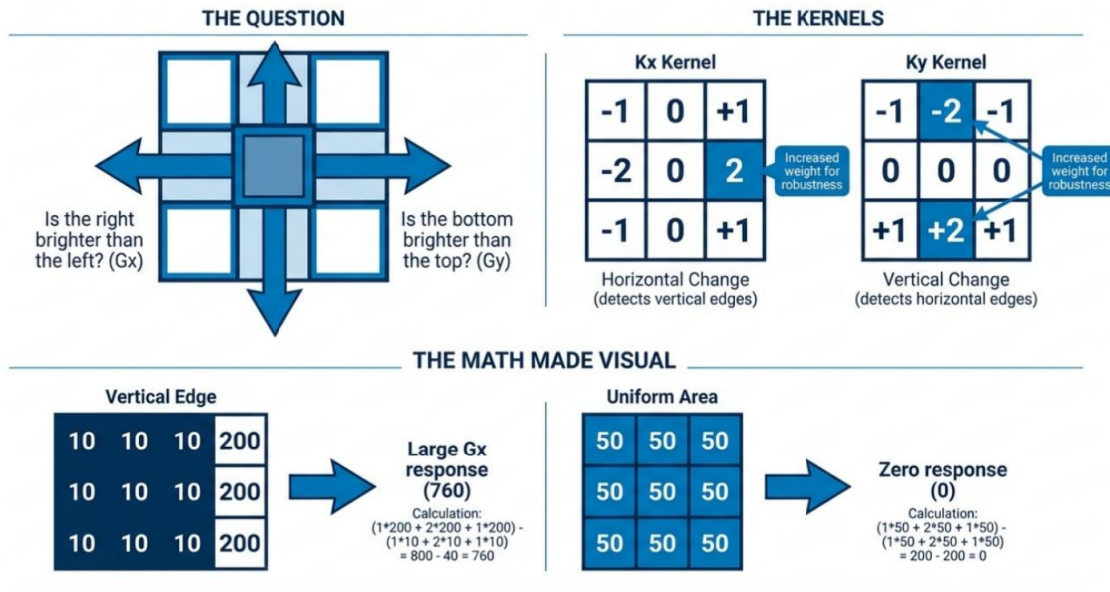
Isolating structural information to simplify image interpretation and feature extraction.

3.1 Sobel Edge Detection

Sobel edge detection is a basic gradient-based method. It applies convolution with two 3x3 kernels to calculate the gradient in the horizontal and vertical directions. The magnitude of the gradient represents the strength of the edge, and its direction indicates the edge's orientation.

SOBEL EDGE DETECTION OPERATOR

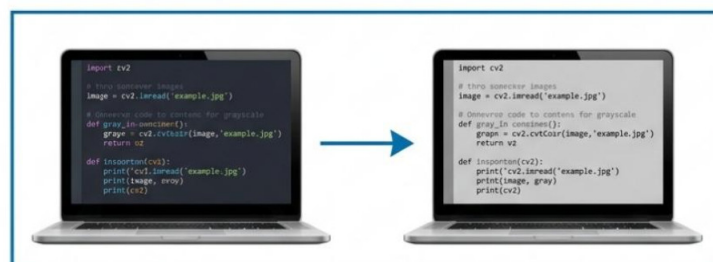




Sobel = A directional difference operation. It measures the intensity gradient to identify where boundaries occur.

Talha Shahid

STEP 1: PREPARATION



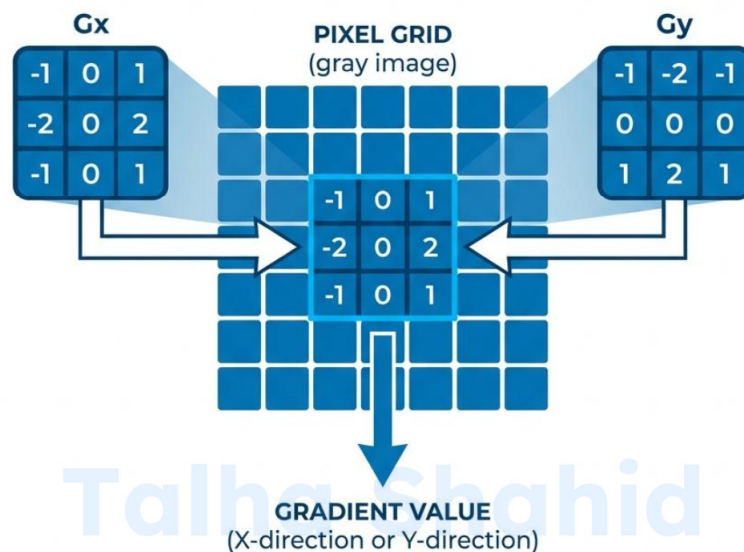
```
gray = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
```

Computer vision tasks are often performed on grayscale images to simplify intensity analysis.

STEP 2: DIRECTIONAL GRADIENTS

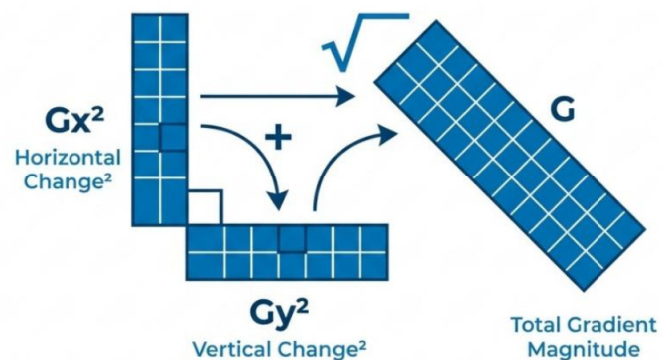
```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

We use 64-bit floats (**CV_64F**) to preserve negative values, which indicate the direction of intensity change.



STEP 3: GRADIENT MAGNITUDE

Combining directional changes gives us the total edge strength at every pixel.

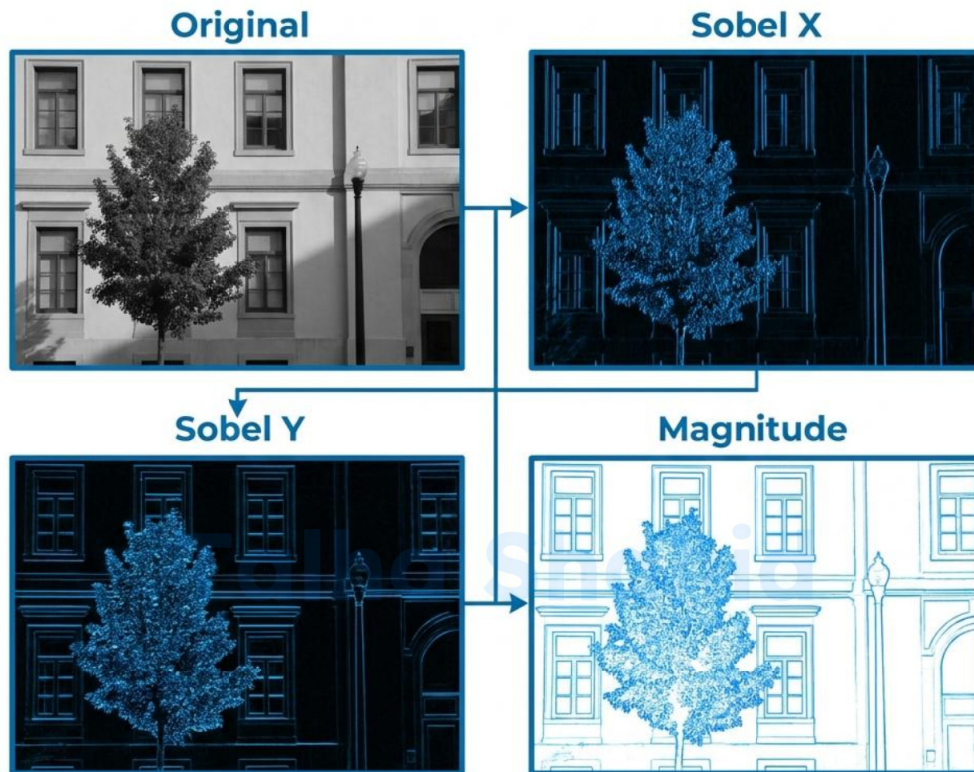


$$G = \sqrt{Gx^2 + Gy^2}$$

```
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
```

STEP 4: OUTPUT

The final Edge Magnitude provides a clear map of all boundaries within the scene.



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("image.jpeg")

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel in X direction
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

```
# Sobel in Y direction
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

# Calculate gradient magnitude
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)

# Convert magnitude to uint8 for display
magnitude = np.uint8(np.clip(magnitude, 0, 255))

# Plot results
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(magnitude, cmap="gray")
plt.title("Sobel Edge Magnitude")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Original Image



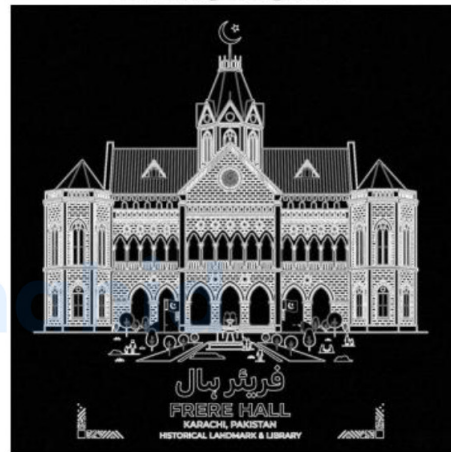
Sobel X



Sobel Y



Sobel Edge Magnitude



3.2 Canny Edge Detection

Canny edge detection is a more sophisticated method that consists of multiple steps

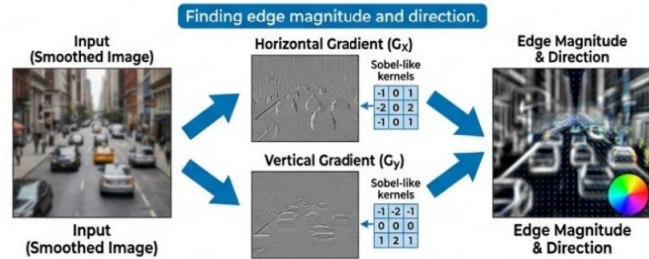
- **Gaussian smoothing:** Reduces noise in the image.
- **Gradient calculation:** Calculates the gradient magnitude and direction.
- **Non-maximum suppression:** Eliminates weak edge pixels.
- **Hysteresis thresholding:** Tracks edges by linking strong edge pixels.

The Canny detector is known for its high accuracy and ability to detect thin and continuous edges.

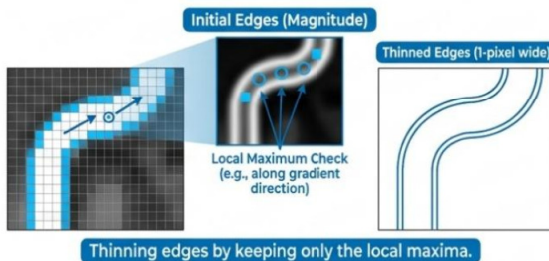
1. GAUSSIAN SMOOTHING



2. GRADIENT CALCULATION

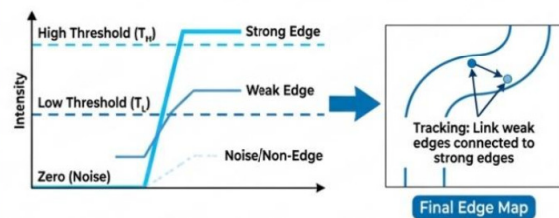


3. NON-MAXIMUM SUPPRESSION

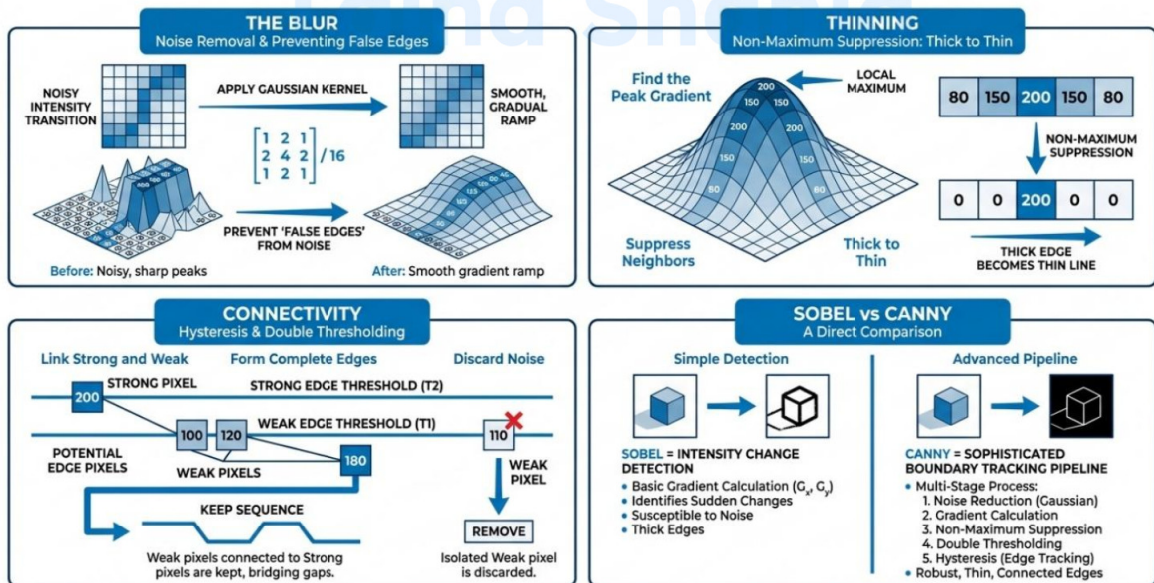


4. HYSTERESIS THRESHOLDING

Final tracking by linking pixels between thresholds.



DEEP INTUITION OF CANNY EDGE DETECTION



```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("image.jpg")

# Convert BGR to RGB for Matplotlib
```



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian Blur
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)

# Apply Canny Edge Detection
edges = cv2.Canny(
    blurred,
    50,      # Lower threshold
    150      # Upper threshold
)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```



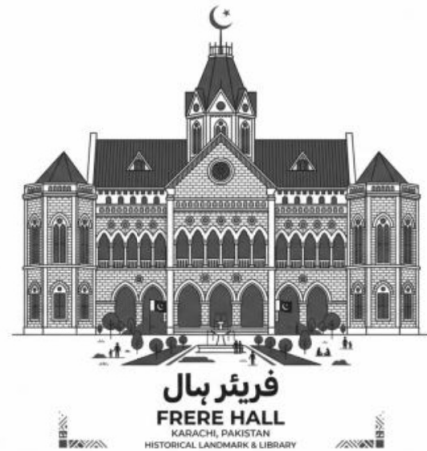
National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

Original Image



Grayscale



Gaussian Blurred



Canny Edges



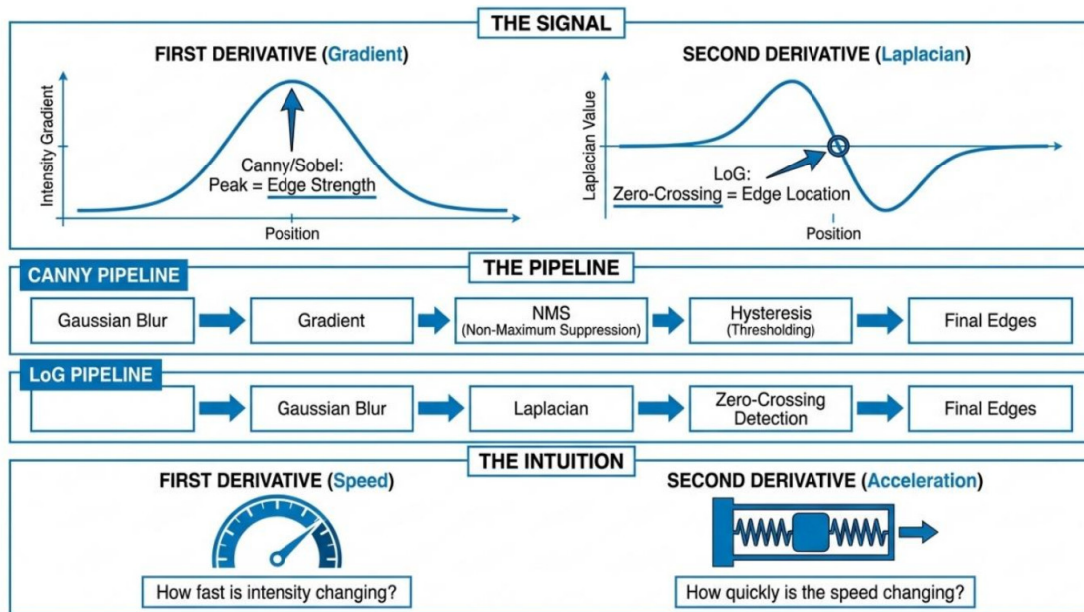
3.3 Laplacian of Gaussian (LoG)

The LoG operator is used to detect edges by first applying a Gaussian smoothing filter to the image and then computing the Laplacian of the smoothed image. The zero-crossings of the resulting image indicate the edges. The LoG operator is effective at detecting edges with varying scales.



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("image.jpg")

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 1: Gaussian Blur
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)

# Step 2: Apply Laplacian
laplacian = cv2.Laplacian(
    blurred,
    cv2.CV_64F
)

# Take absolute values for visualization
laplacian_abs = cv2.convertScaleAbs(laplacian)

# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
```



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

```
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

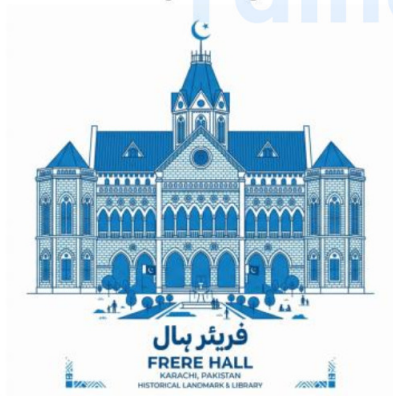
plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred")
plt.axis("off")

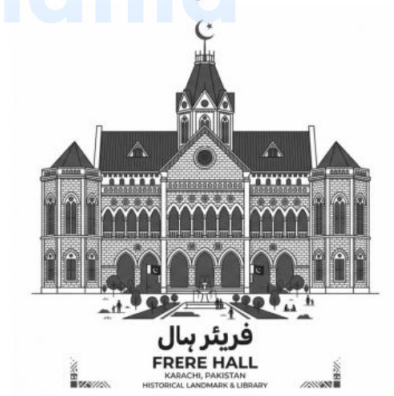
plt.subplot(2, 2, 4)
plt.imshow(laplacian_abs, cmap="gray")
plt.title("Laplacian of Gaussian (LoG)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

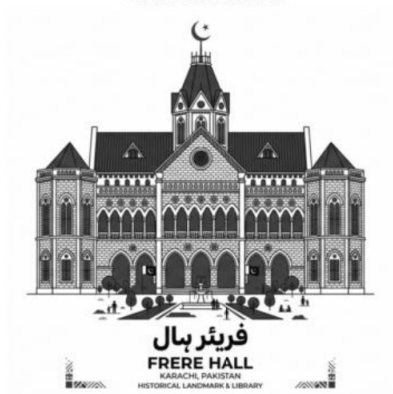
Original Image



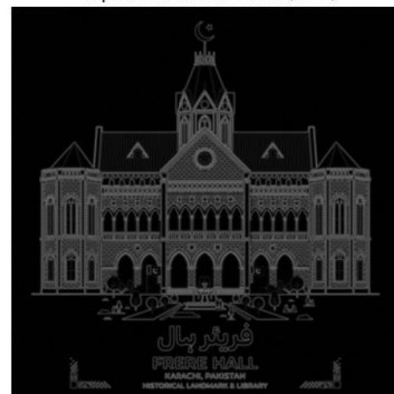
Grayscale



Gaussian Blurred



Laplacian of Gaussian (LoG)





4. Introduction to Hough Transformation

The Hough Transform is a fundamental technique in computer vision and image processing used to detect simple shapes within an image, most commonly lines and circles. It was first introduced by Paul Hough in 1962 for detecting lines in binary images, and later extended to detect other shapes like circles and ellipses. The Hough Transform is particularly useful when traditional methods like gradient-based edge detection are insufficient for detecting specific shapes. It's widely employed in various computer vision applications, such as image analysis, object recognition, and feature extraction.

The Hough Transform works by transforming the spatial domain of an image into a parameter space, where each pixel in the image corresponds to a curve or a point in this parameter space. The primary purpose of this transformation is to identify patterns that are represented as lines or curves in the parameter space.

4.1 Hough Transform for Line Detection

Parametric Representation of Lines: The Hough Transform starts with the parametric representation of lines, where a line in Cartesian coordinates (x, y) is represented as

$$y = mx + b$$

Here, 'm' is the slope of the line, and 'b' is the y-intercept

- **Parameter Space:** In the Hough Transform, we transform this representation into a parameter space with 'm' and 'b' as axes. Each line in the image corresponds to a point in this parameter space.
- **Voting:** For each edge pixel (typically obtained from edge detection methods like Canny), we calculate the 'm' and 'b' values that could generate the line passing through that pixel. This is essentially a voting process where we increment the corresponding (m, b) cell in the parameter space.
- **Peak Detection:** After processing all edge pixels, the parameter space is examined to find peaks. These peaks represent the lines in the image.
- **Back-Transformation:** Once the lines are detected in the parameter space, they can be back-transformed into the image space, revealing the position and orientation of the lines.



4.2 Hough Transform for Circle Detection

Parametric Representation of Circles: In the case of circle detection, a circle in Cartesian coordinates (x, y) is represented as

$$(x - a)^2 + (y - b)^2 = r^2$$

Here, (a, b) represents the center of the circle, and 'r' is its radius.

- **Parameter Space:** The parameter space for circle detection has three dimensions, corresponding to 'a,' 'b,' and 'r.'
- **Voting:** Similar to line detection, we vote for potential circle centers (a, b) and radii (r) based on edge pixel information in the image.
- **Peak Detection:** After processing all edge pixels, peaks in the parameter space correspond to potential circle centers and radii.
- **Back-Transformation:** Detected circles in the parameter space can be back-transformed into the image space, revealing the position and size of the circles.

5. SIFT Feature Extraction

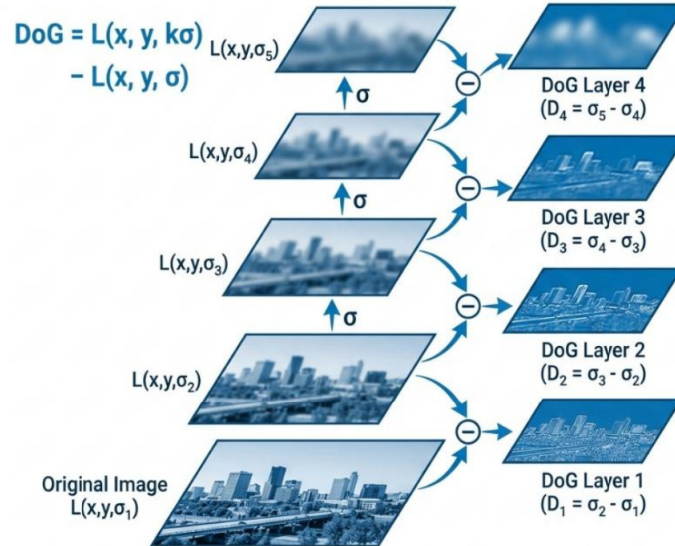
SIFT (Scale-Invariant Feature Transform) is a powerful computer vision technique used for feature extraction and matching in images. It is particularly useful for tasks such as object recognition, image stitching, and tracking.

5.1 Scale-Space Extrema Detection

SIFT starts by detecting keypoint candidates at multiple scales within the image. This is achieved by applying a Difference of Gaussians (DoG) operator to identify local extrema in the scale space. The DoG is calculated by subtracting two Gaussian-blurred versions of the image.

1. SCALE-SPACE EXTREMA DETECTION

Searching across multiple scales ensures we find features whether they are large or small.

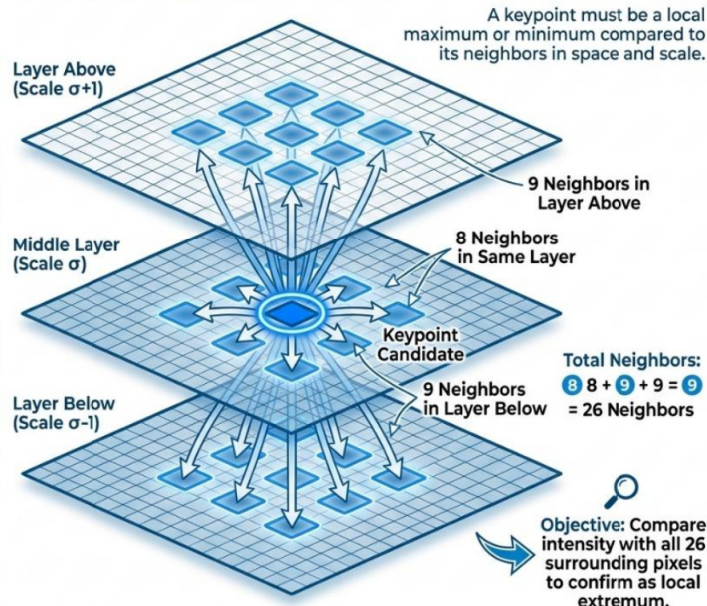


5.2 Keypoint Localization

For each candidate keypoint, SIFT refines the location by fitting a 3D quadratic function to the nearby data points in the scale space. This step helps to accurately localize the keypoints.

2. KEYPOINT LOCALIZATION

Concept: The 26-Neighbor Check



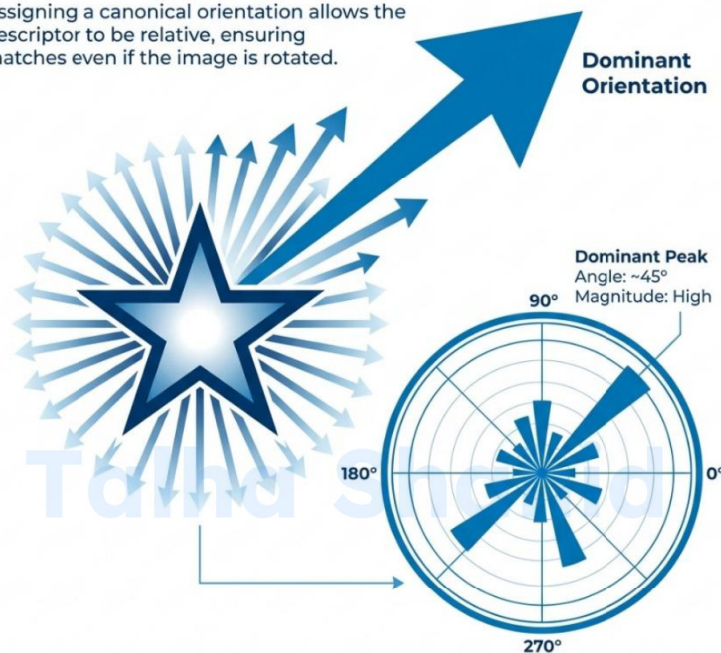
5.3 Orientation Assignment

To make the keypoints invariant to rotation, SIFT assigns an orientation to each keypoint based on the dominant gradient direction in its local neighborhood. This results in a canonical orientation for each keypoint.

3. ORIENTATION ASSIGNMENT

Concept: "Invariance to Rotation"

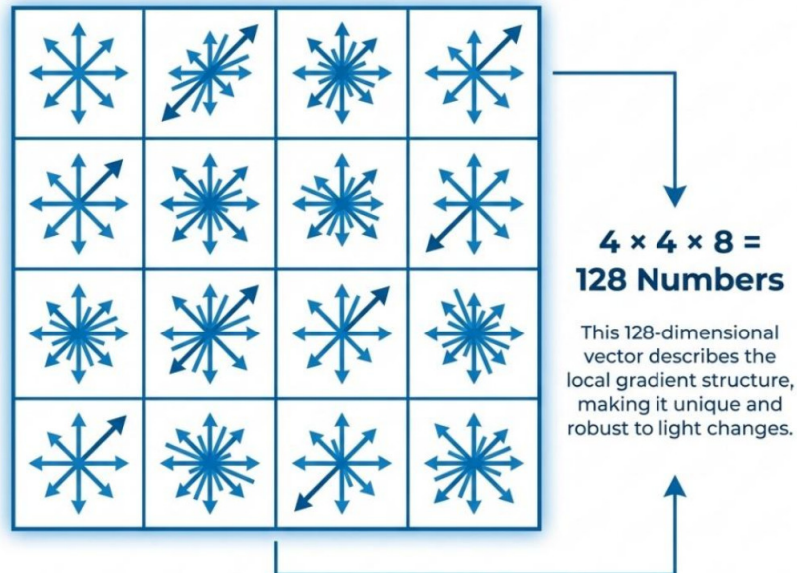
Assigning a canonical orientation allows the descriptor to be relative, ensuring matches even if the image is rotated.



5.4 Keypoint Descriptor Generation

Around each keypoint, SIFT constructs a descriptor that characterizes the local image region. The descriptor is created by considering the gradient magnitudes and orientations in a specific pattern of sub-regions surrounding the keypoint. This pattern is divided into smaller bins, and histograms of gradient orientations are computed for each sub-region. The concatenated histograms form the keypoint descriptor.

4. DESCRIPTOR GENERATION



Talha Shahid

5.5 Keypoint Matching

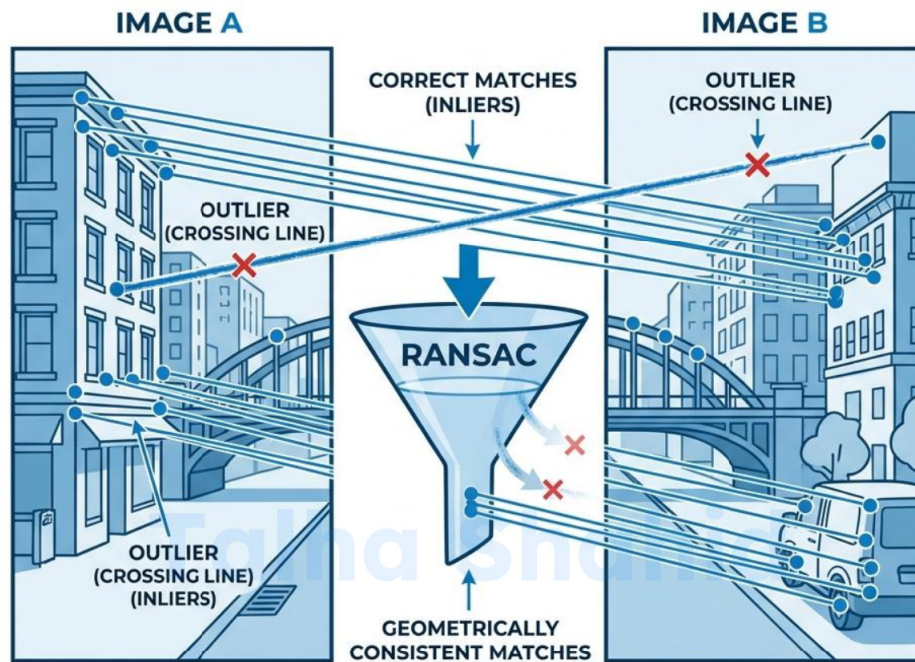
Once the descriptors are computed for keypoints in multiple images, these descriptors can be used to match keypoints between images. A common technique for matching involves calculating the Euclidean distance or other similarity measures between the descriptors of keypoints in different images.

5.6 Outlier Rejection

Keypoint matches may contain some outliers due to variations in lighting, viewpoint changes, or noise. Techniques like RANSAC (Random Sample Consensus) can be employed to robustly estimate transformation models (e.g., affine or perspective transformations) and reject outliers.

5. MATCHING & OUTLIER REJECTION

Concept: Euclidean Distance + RANSAC



Explanation: Descriptors are matched by similarity (e.g., Euclidean Distance), and RANSAC ensures only geometric consistent matches are kept.

5.7 Final Matching and Homography Estimation

After outlier rejection, the remaining keypoint matches are used to estimate the transformation (homography) between the images. This transformation can be used for image stitching, object recognition, or other applications.

```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("image.jpg")

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```



```
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

# Draw keypoints
image_keypoints = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Display
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(image_keypoints)
plt.title("SIFT Keypoints")
plt.axis("off")

plt.tight_layout()
plt.show()

# Print information
print("Number of keypoints:", len(keypoints))
print("Descriptor shape:", descriptors.shape)
```

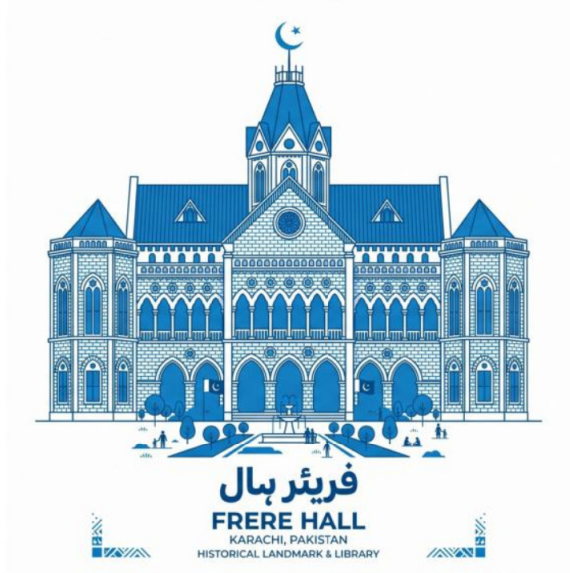


National University of Computer & Emerging Sciences -

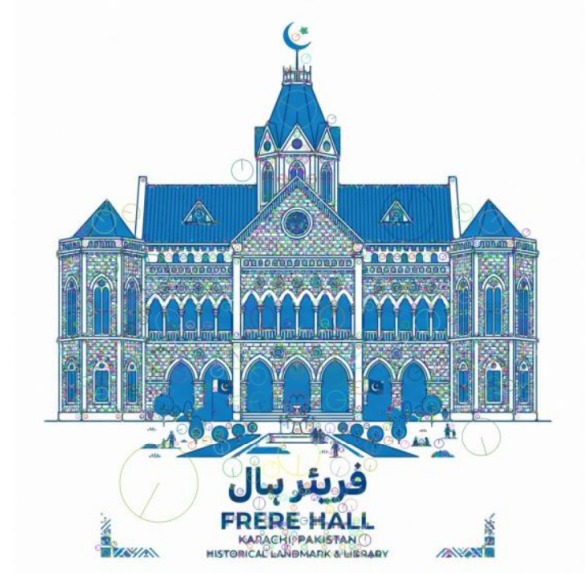
NUCES - Karachi

Department of Artificial Intelligence

Original Image



SIFT Keypoints



Talha Shahid